Algoritmi e Strutture Dati

Capitolo 2
Modelli di calcolo e metodologie di analisi

Camil Demetrescu, Irene Finocchi,
Giuseppe F. Italiano



Analisi di algoritmi ricorsivi

- La quantità di risorsa usata da algoritmi ricorsivi può essere espressa tramite relazioni di ricorrenza
- Risolvibili tramite vari metodi generali:

  iterazione, sostituzione, alberi di ricorsione, teorema Master...

Copyright © 2008 - The McGraw - Hill Companies, srl



Esempio di algoritmo ricorsivo

L'algoritmo di ricerca binaria può essere riscritto in forma ricorsiva come

algoritmo `ricercaBinariaRic`$(array\ L,\ elemento\ x) \rightarrow booleano$

1. $n \leftarrow$ lunghezza di L
2. **if** $(\ n = 0\ )$ **then return** non trovato
3. $i \leftarrow \lceil n/2 \rceil$
4. **else if** $(\ L[i] = x\ )$ **then return** trovato
5. **else if** $(\ L[i] > x\ )$ **then return** `ricercaBinariaRic`$(x, L[1; i-1])$
6. **else return** `ricercaBinariaRic`$(x, L[i+1; n])$

(Si assume L ordinato e indicizzato da 1 a n)

Copyright © 2008 - The McGraw - Hill Companies, srl



Esempio di algoritmo ricorsivo

algoritmo `ricercaBinariaRic`$(array\ L,\ elemento\ x) \rightarrow booleano$

1. $n \leftarrow$ lunghezza di L
2. **if** $(\ n = 0\ )$ **then return** non trovato
3. $i \leftarrow \lceil n/2 \rceil$
4. **else if** $(\ L[i] = x\ )$ **then return** trovato
5. **else if** $(\ L[i] > x\ )$ **then return** `ricercaBinariaRic`$(x, L[1; i-1])$
6. **else return** `ricercaBinariaRic`$(x, L[i+1; n])$

Come analizzarlo?

Copyright © 2008 - The McGraw - Hill Companies, srl



Equazioni di ricorrenza

Il tempo di esecuzione della ricerca binaria (versione ricorsiva) può essere descritto tramite l'equazione di ricorrenza:

$$T(n) = \begin{cases} 1 & \text{se } n=1 \text{ (passo base)} \\ c + T(\lceil n/2 \rceil) & \text{se } n>1 \end{cases}$$

O semplicemente (assumendo n potenza del 2):

$$T(n) = \begin{cases} 1 & \text{se } n=1 \text{ (passo base)} \\ c + T(n/2) & \text{se } n>1 \end{cases}$$

Copyright © 2008 - The McGraw - Hill Companies, srl



Metodo dell'iterazione

- Iterare la ricorsione, ottenendo una sommatoria dipendente solo dalla dimensione n e dalle condizioni iniziali.
- Le tecniche per limitare le sommatorie possono essere usate poi per fornire limiti alla soluzione.

Esempio (Ricerca binaria) : $T(n) = c + T(n/2)$

$T(n/2) = c + T(n/4)$ $T(n/4) = c + T(n/8)$ ...

$T(n) = c + T(n/2) = 2c + T(n/4) = 3c + T(n/8) = \ldots$
$= (\sum_{j=1\ldots i} c) + T(n/2^i) = i\,c + T(\mathbf{n/2^i})$

Raggiungiamo il passo base per $n/2^i = 1$ e cioè per $i = \lg n$, da cui: $T(n) = c \lg n + T(1) = O(\lg n)$

Copyright © 2008 - The McGraw - Hill Companies, srl



Metodo dell'iterazione

- Può richiedere l'uso di molta algebra e calcoli complessi
- Bisogna concentrarsi su due parametri:
  - il numero di volte che la ricorrenza deve essere iterata per raggiungere le condizioni iniziali (a contorno)
  - la somma dei termini originati da ogni livello del processo di iterazione
- Talvolta nel processo di iterazione si può indovinare la soluzione senza fare tutti i calcoli matematici. In tal caso è possibile passare al metodo di sostituzione.

Copyright © 2008 - The McGraw - Hill Companies, srl



Metodo della sostituzione

Idea: "indovinare" una soluzione, ed usare induzione matematica per provare che la soluzione dell'equazione di ricorrenza è effettivamente quella intuita

Esempio: $T(n) = n + T(n/2)$, $T(1)=1$

Assumiamo che la soluzione sia $T(n) \leq c\,n$ per una costante c opportuna

Passo base: $T(1)=1 \leq c\,1$ per ogni $c \geq 1$

Passo induttivo: $T(n)= n + T(n/2) \leq n+c\,(n/2) = (1+c/2)n$

da cui $T(n) \leq c\,n$ per $1+c/2 <= c$ ovvero per $c \geq 2$

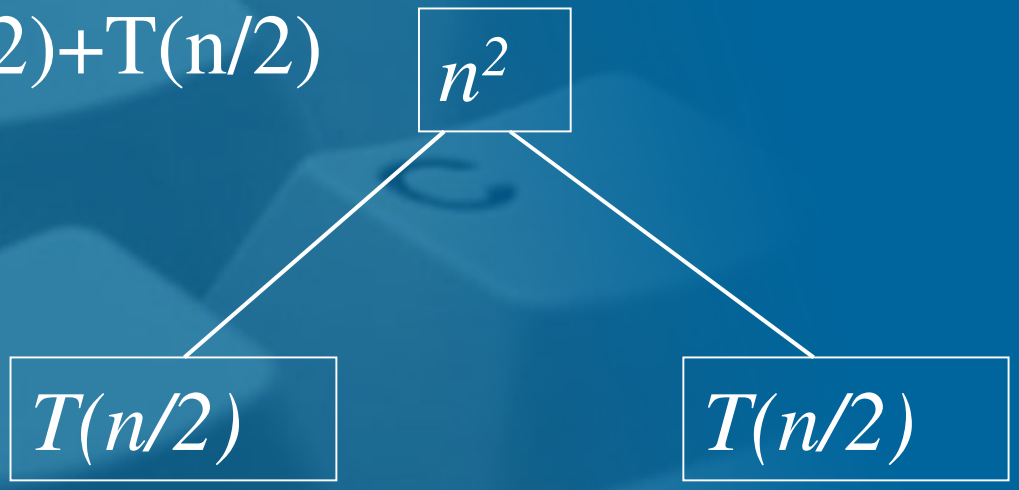
Copyright © 2008 - The McGraw - Hill Companies, srl

Alberi di ricorsione

Un albero di ricorsione è un metodo molto comodo per visualizzare tutto ciò che accade quando si itera una ricorrenza.

Consideriamo la ricorrenza:

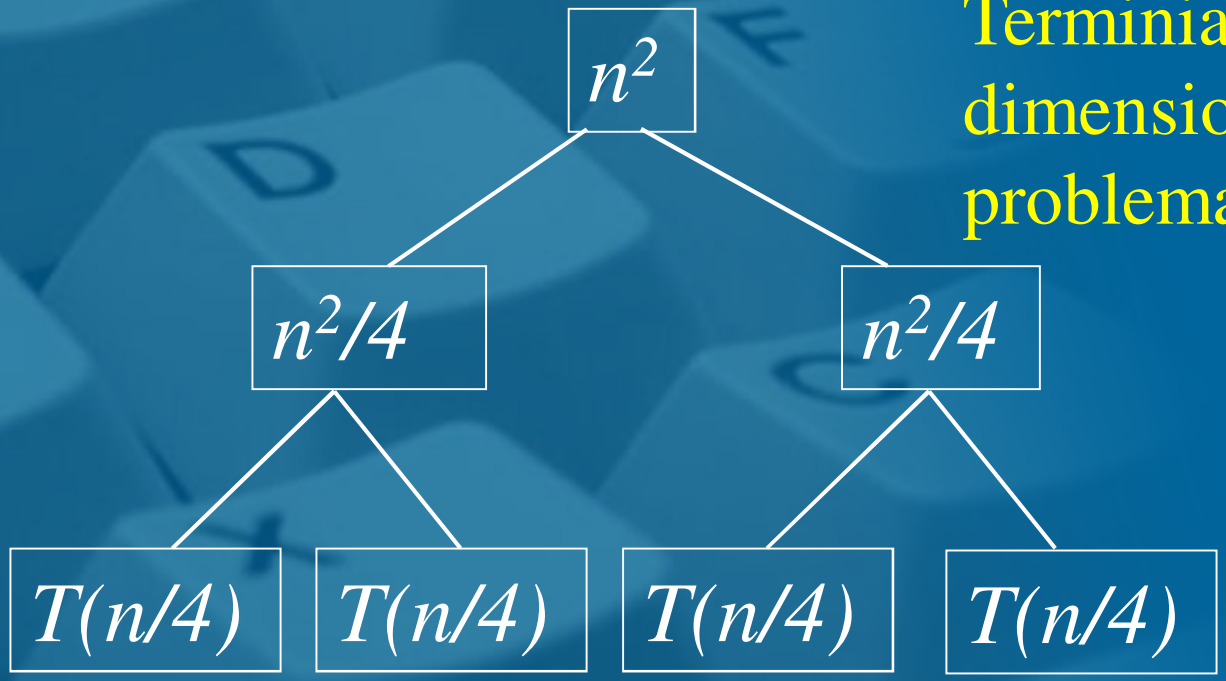
$T(n) = n^2 + T(n/2) + T(n/2)$



Alberi di ricorsione

$T(n)= n^2 +T(n/2)+T(n/2)$

$T(n/2)=n^2/4+T(n/4)+T(n/4)$



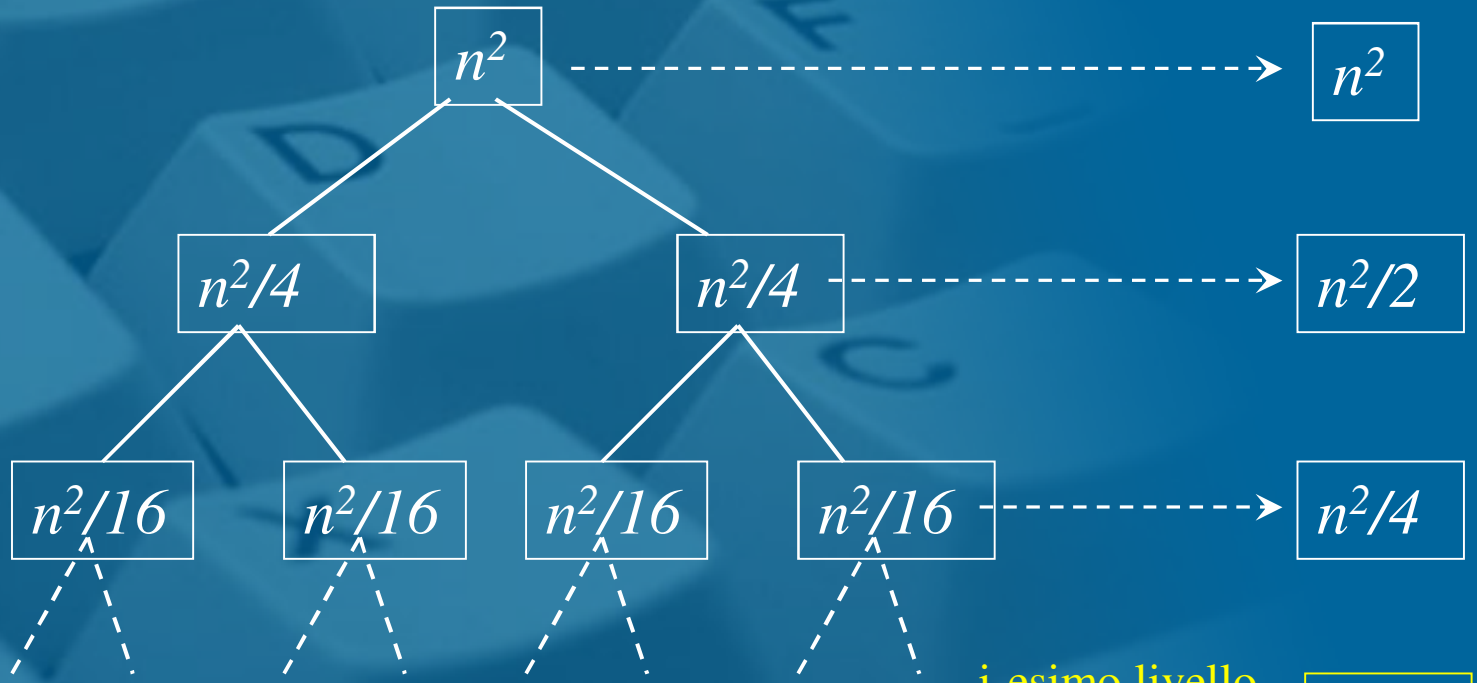
I nodi riportano le dimensioni dei sotto-problemi

Terminiamo quando la dimensione del sotto-problema è 1: $n/2^i = 1$

Alberi di ricorsione

$$T(n)=n^2+T(n/2)+T(n/2)$$

Si analizza il totale delle dimensioni per livelli





Alberi di ricorsione

Poiché il valore decresce geometricamente, il valore totale differisce, al più di un fattore costante dal termine più grande, quindi la soluzione è $\Theta(n^2)$

$$\frac{n}{2^i} = 1 \rightarrow i = \lg n$$

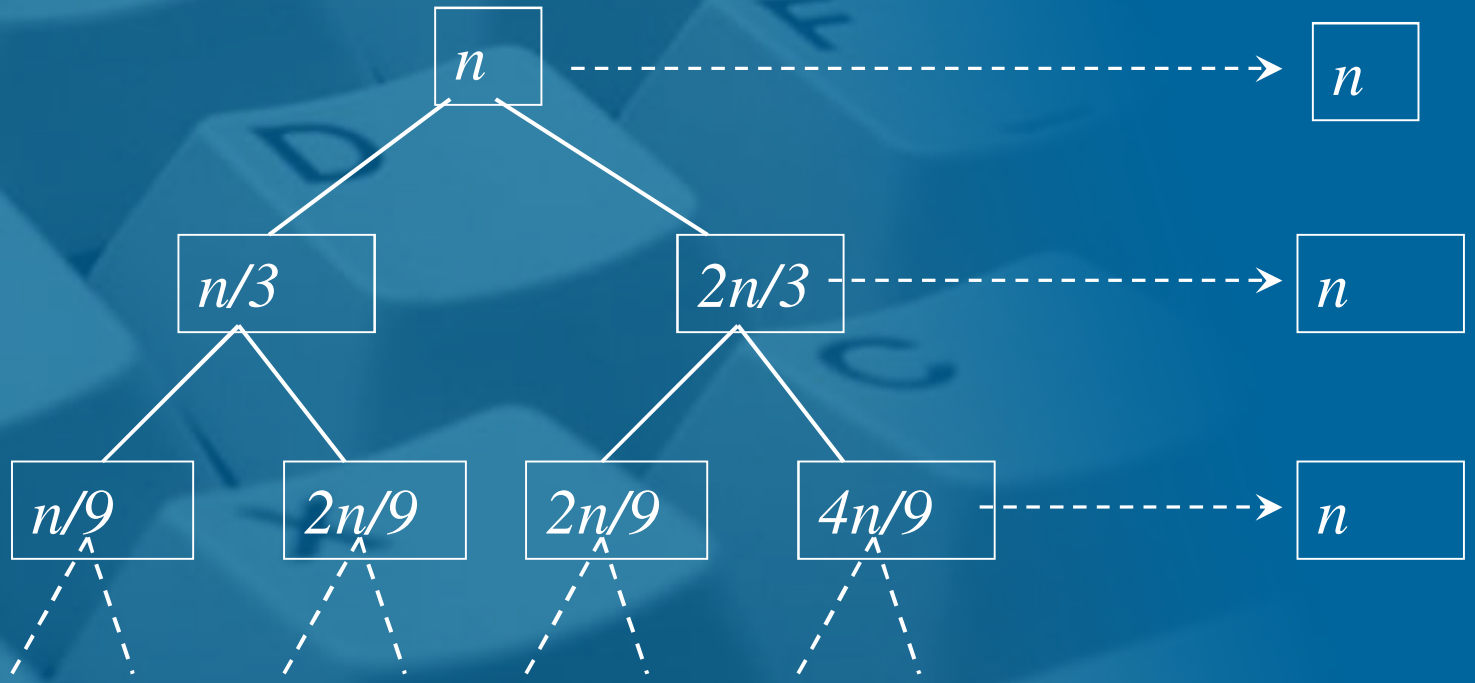
dimensione
del sotto-problema
pari alla dimensione minima 1

$$\begin{aligned} T(n) &= \sum_{i=0}^{\lg n} \frac{n^2}{2^i} \\ &= n^2 \sum_{i=0}^{\lg n} \left(\frac{1}{2^i}\right) = n^2 \frac{1-(1/2)^{\lg n+1}}{1/2} \\ &= n^2\left(2-(1/2)^{\lg n}\right) = n^2\left(2-n^{\lg(1/2)}\right) \\ &= 2n^2 - n \\ &= \Theta(n^2) \end{aligned}$$

Alberi di ricorsione

Un altro esempio

$T(n)=T(n/3)+T(2n/3)+n$





Alberi di ricorsione

Il ramo destro dell'albero è quello più lungo, quindi:

$$\left(\frac{2}{3}\right)^i n = 1 \quad \rightarrow \quad i = \log_{3/2} n$$

$$T(n) \leq \sum_{i=0}^{\log_{3/2} n} n$$

$$= n \log_{3/2} n$$

$$= O(n \lg n)$$



Teorema Master

Permette di analizzare algoritmi basati sulla tecnica del *divide et impera*:

- dividi il problema (di dimensione n) in a sottoproblemi di dimensione n/b con a >= 1 e b > 1
- risolvi i sottoproblemi ricorsivamente
- ricombina le soluzioni

Sia f(n) il tempo per dividere e ricombinare istanze di dimensione n. La relazione di ricorrenza è data da:

$$T(n) = \begin{cases} a\,T(n/b) + f(n) & \text{se } n>1 \\ 1 & \text{se } n=1 \end{cases}$$

Copyright © 2008 - The McGraw - Hill Companies, srl



Teorema Master

La relazione di ricorrenza:

$$T(n) = \begin{cases} a\,T(n/b) + f(n) & \text{se } n>1 \\ 1 & \text{se } n=1 \end{cases}$$

ha soluzione:

1. $T(n) = \Theta(n^{\log_b a})$ se $f(n)=O(n^{\log_b a-\varepsilon})$ per $\varepsilon>0$
2. $T(n) = \Theta(n^{\log_b a} \log n)$ se $f(n) = \Theta(n^{\log_b a})$
3. $T(n) = \Theta(f(n))$ se $f(n)=\Omega(n^{\log_b a+\varepsilon})$ per $\varepsilon>0$ e $a\,f(n/b)\leq c\,f(n)$ per $c<1$ e n sufficientemente grande

Copyright © 2008 - The McGraw - Hill Companies, srl



Esempi

1) $T(n) = n + 2T(n/2)$

   $a=2,\ b=2,\ f(n)=n=\Theta(n^{\log_2 2})$ ➡ $T(n)=\Theta(n \log n)$

   (caso 2 del teorema master)

2) $T(n) = c + 3T(n/9)$

   $a=3,\ b=9,\ f(n)=c=O(n^{\log_9 3 - \varepsilon})$ ➡ $T(n)=\Theta(\sqrt{n})$

   (caso 1 del teorema master) per $\varepsilon = 0{,}5$

3) $T(n) = n + 3T(n/9)$

   $a=3,\ b=9,\ f(n)=n=\Omega(n^{\log_9 3 + \varepsilon})$

   $3(n/9) \leq c\, n$ per $c=1/3$ e $\varepsilon = 0{,}5$ ➡ $T(n)=\Theta(n)$

   (caso 3 del teorema master)

Copyright © 2008 - The McGraw - Hill Companies, srl



Esercizio (appello 31 Ottobre 2021)

- Calcolare la complessità tempo della seguente funzione ricorsiva, assumendo che l'algoritmo MergeSort ha complessità Θ(n log n)

```
foo( n,A ){  //n intero positivo e A array di dimensione n
        if (n < 10) return 1;
        else if (n < 1234){
                tmp = n;
                for (i = 1; i <= n; i = i*2)
                  for (j = 1; j <= n; j = j+1)
                        tmp = tmp + i * j;
                }          return tmp;
        else  {  tmp = n;  MergeSort(A,0,n-1); }
        return tmp + foo( n/4 ) + foo( n/4 ) + foo( n/4 );
}
```


Copyright © 2008 - The McGraw - Hill Companies, srl



Riepilogo

- Esprimiamo la quantità di una certa risorsa di calcolo (tempo, spazio) usata da un algoritmo in funzione della dimensione n dell'istanza di ingresso
- La notazione asintotica permette di esprimere la quantità di risorsa usata dall'algoritmo in modo sintetico, ignorando dettagli non influenti
- A parità di dimensione n, la quantità di risorsa usata può essere diversa, da cui la necessità di analizzare il caso peggiore o, se possibile, il caso medio
- La quantità di risorsa usata da algoritmi ricorsivi può essere espressa tramite relazioni di ricorrenza, risolvibili tramite vari metodi generali

Copyright © 2008 - The McGraw - Hill Companies, srl